

GUÍA PARA LA DOCUMENTACIÓN PROFESIONAL DE SOFTWARE

Programa: Ingeniería de Software

Nivel: Quinto Semestre

1. Introducción

La documentación de software es un componente esencial en el desarrollo de sistemas de información. No se trata únicamente de un requisito académico, sino de una práctica profesional que garantiza mantenibilidad, escalabilidad y sostenibilidad en el tiempo.

En entornos empresariales, la ausencia de documentación adecuada genera dependencia de personas, aumento de la deuda técnica y dificultades en procesos de mantenimiento o evolución del sistema.

Un sistema de software no se considera completo únicamente cuando funciona, sino cuando puede ser comprendido, mantenido y evolucionado por otros profesionales.

2. Objetivos de la Documentación

La documentación de software debe cumplir los siguientes objetivos:

- Explicar el propósito y alcance del sistema.
- Describir los requerimientos funcionales y no funcionales.
- Justificar las decisiones arquitectónicas.
- Facilitar la incorporación de nuevos desarrolladores.
- Permitir la operación y mantenimiento del sistema.
- Servir como respaldo técnico ante auditorías o revisiones.

3. Principios de una Buena Documentación

Una documentación profesional debe cumplir con los siguientes principios:

3.1 Claridad

Debe ser comprensible para cualquier miembro del equipo, incluso si no participó en el desarrollo.

3.2 Coherencia

La información documentada debe coincidir con la implementación real del sistema.

3.3 Actualización

Debe mantenerse alineada con los cambios del proyecto.

3.4 Justificación Técnica

No solo debe indicar qué se hizo, sino también por qué se tomó cada decisión.

3.5 Proporcionalidad

Debe documentarse lo relevante. Ni exceso innecesario ni ausencia de información crítica.

4. Estructura Recomendada de Documentación

A continuación se presenta una estructura profesional recomendada para proyectos académicos con enfoque empresarial.

4.1 Documento de Visión

Define el propósito general del sistema.

Debe incluir:

- Introducción
- Problema que se busca resolver
- Objetivos del sistema
- Alcance
- Actores o stakeholders
- Restricciones técnicas
- Supuestos del proyecto

4.2 Especificación de Requerimientos

4.2.1 Requerimientos Funcionales

Deben describir las funcionalidades del sistema. Se recomienda estructurarlos en formato tabular:

- Identificador
- Nombre

- Descripción
- Prioridad
- Criterios de aceptación

Ejemplo:

RF-01 – El sistema debe permitir la autenticación de usuarios mediante credenciales válidas.

4.2.2 Requerimientos No Funcionales

Describen características de calidad del sistema:

- Rendimiento
- Seguridad
- Disponibilidad
- Escalabilidad
- Usabilidad
- Compatibilidad
- Mantenibilidad

Ejemplo:

RNF-01 – El tiempo de respuesta del sistema no debe superar los 2 segundos en condiciones normales de operación.

4.3 Documentación de Arquitectura

Debe describir la estructura técnica del sistema.

Se recomienda incluir:

- Diagrama de contexto
- Diagrama de contenedores
- Diagrama de componentes
- Diagrama de despliegue

Además, debe explicarse:

- Patrón arquitectónico utilizado (MVC, Clean Architecture, Hexagonal, etc.)
- Justificación de la arquitectura seleccionada
- Tecnologías empleadas

4.4 Registro de Decisiones Arquitectónicas (ADR)

Las decisiones técnicas relevantes deben documentarse formalmente.

Cada registro debe incluir:

- Decisión tomada
- Problema que resuelve
- Alternativas evaluadas
- Justificación de la elección
- Impacto en el sistema

Este tipo de documentación evidencia pensamiento ingenieril y criterio técnico.

4.5 Documentación Técnica del Proyecto

Debe incluir un archivo README estructurado que contenga:

- Descripción general del proyecto
- Requisitos de instalación
- Dependencias
- Variables de entorno
- Instrucciones de ejecución
- Instrucciones para pruebas
- Estructura del proyecto

La documentación técnica debe facilitar que cualquier desarrollador pueda levantar el proyecto sin dificultad.

4.6 Documentación de APIs

Cuando el sistema expone servicios, debe documentarse cada endpoint indicando:

- Método HTTP
- Ruta
- Parámetros
- Cuerpo de solicitud
- Respuesta esperada
- Códigos de estado
- Manejo de errores

Se recomienda el uso de herramientas como OpenAPI o Swagger.

4.7 Manual Técnico

Dirigido a personal técnico o de soporte.

Debe incluir:

- Arquitectura general
- Configuración de despliegue
- Gestión de logs
- Procedimientos de respaldo
- Solución de problemas comunes

4.8 Manual de Usuario

Dirigido a usuarios finales. Debe utilizar lenguaje no técnico.

Debe incluir:

- Acceso al sistema
- Funcionalidades principales
- Flujos de uso
- Preguntas frecuentes
- Manejo de errores comunes

4.9 Documentación de Pruebas

Debe contemplar:

- Plan de pruebas
- Casos de prueba
- Evidencias de ejecución
- Resultados obtenidos
- Incidencias encontradas

La documentación de pruebas demuestra control de calidad y disciplina técnica.

4.10 Control de Versiones

Debe documentarse:

- Estrategia de ramas (GitFlow o similar)
- Convención de commits
- Versionamiento semántico
- Procedimiento de integración

5. Herramientas Recomendadas

- Markdown
- GitHub o GitLab
- Draw.io o herramientas UML
- Swagger / OpenAPI
- Confluence o Wiki de proyecto
- PlantUML

La elección de herramientas debe responder a la naturaleza del proyecto y al entorno de trabajo.

6. Errores Comunes en la Documentación

- Documentar únicamente al final del proyecto.
- Copiar plantillas sin análisis.
- No actualizar diagramas.
- No justificar decisiones técnicas.
- No documentar dependencias.
- Documentación desconectada del código real.

7. Conclusión

La documentación no es un requisito académico aislado, sino una práctica esencial en el ejercicio profesional de la ingeniería de software. Un desarrollador competente no solo escribe código funcional, sino que estructura, explica y respalda técnicamente sus decisiones.

La calidad de la documentación refleja el nivel de madurez del equipo y la solidez del proceso de desarrollo.

Un proyecto bien documentado es un proyecto que puede mantenerse, escalar y evolucionar.